MobiFlight Connector Setup for Three Gauges

C172 ESP Hub | MobiFlight + ESP-NOW

Current hub: flashed firmware 0.0.1; USB port COM22; USB ID 303A:17E0. Module serial SN-D00FDC1D0470. Verified on 13 October 2026. The port may change after reconnection.

The ESP32-S3 hub is the MobiFlight module. The encoder devices are fake registration devices on the hub; the real encoders are attached to the remote display ESP32 boards and arrive over ESP-NOW.

Fake Encoder Pins

For a Waveshare ESP32-S3-Zero / S3 Mini-style hub, start with:

MobiFlight encoder name	Fake pin A	Fake pin B
`ALT_BARO`	GPIO1	GPIO2
`AIRSPEED_TAPE`	GPIO4	GPIO5
`ATTITUDE_REF`	GPIO6	GPIO7

Do not wire anything to these pins for the first proof.

Avoid `GPIO0`, `GPIO19`, `GPIO20`, and any board-specific onboard hardware pins.

Required Outputs

Add the `C172_ESP_HUB` custom output device and configure these simulator values:

Create one custom device instance named `SIXPACK` and route every output below through that same instance. Enable all five required simulator outputs.

ID	Message	MSFS custom RPN source	Units
0	`ALTITUDE`	`(A:INDICATED ALTITUDE:1, feet)`	feet
1	`BARO`	`(A:KOHLSMAN SETTING HG:1, inHg)`	inHg
2	`AIRSPEED`	`(A:AIRSPEED INDICATED, knots)`	knots
3	`PITCH`	`(A:PLANE PITCH DEGREES, degrees) -1 *`	degrees, nose up positive
4	`BANK`	`(A:PLANE BANK DEGREES, degrees)`	degrees
5	`AIRSPEED_TAPE_ANGLE`	`0` initially	degrees, optional
6	`ATTITUDE_REF_OFFSET`	`0` initially	display pixels, optional

The renderer moves the horizon down for positive pitch. The starting expression negates simulator pitch; verify nose-up direction on the bench and adjust the transform if the selected aircraft/source differs. Verify bank direction against the simulator during the bench test before adjusting any transform. The core SimVar names/units are documented in the MSFS SDK. Use the altimeter index for the instrument you want to mirror (the table starts with

index 1). Keep decimal precision: altitude 1, baro 2, airspeed/pitch/bank 2 places are reasonable starting settings. Avoid integer rounding of baro. The optional values start at zero and can later be driven by the aircraft's actual reference-marker variables; do not invent aircraft LVars for them.

The displays should render from these values only. Encoder actions should go:

```
display encoder -> ESP-NOW -> hub -> MobiFlight input event -> MSFS -> MobiFlight output -> hub ->
ESP-NOW -> display
```

This keeps the sim and displays synchronized.

Build and Bench Test (Windows)

From the repository root:

```
git submodule update --init third_party/MobiFlight-FirmwareSource
./Software/Hub/mobiflight/build_hub.ps1 -PlatformIO
"$env:USERPROFILE/.platformio/penv/Scripts/platformio.exe"
```

The helper copies the hub package into the firmware submodule and updates its local `platformio.ini`; these generated submodule changes are expected, and must not be committed upstream. Flash the resulting hub `.bin` via PlatformIO with `run -e c172_esp_hub_esp32s3 -t upload --upload-port COMx` from that submodule directory, using the actual hub port.

The hub USB stream belongs exclusively to MobiFlight CmdMessenger. Close serial monitors before connecting Connector. Install the board/device definitions from this folder in the corresponding Connector definition folders. The board definition matches the hub USB VID/PID (`303A:17E0`), exposes one custom device and three optional encoders, and advertises zero LED outputs. It disables Connector firmware installation: flash through PlatformIO. Import `C172 ESP Hub.mfmc` for the output-only module configuration, then add the five required simulator output mappings above. Board discovery and the configuration import still require hardware validation. Register exactly one `C172_ESP_HUB` custom device for the three gauges. The data path does not require the fake encoders until testing remote inputs. No standby battery or ELT LED outputs or pins are needed.

Build and flash `sixpack_airspeed_espnow`, `sixpack_attitude_espnow`, and `sixpack_altimeter_espnow` from `Software`. All share radio channel 1, without joining an access point. Broadcast state works without hard-coded client MACs. The hub publishes the aggregate protocol-v1 state at 25 Hz; each client renders only its own fields at its existing frame interval.

1. Start Connector with the five required outputs. Compare all three gauges against the cockpit, including baro decimals and pitch/bank direction.

2. Change one simulator value at a time; confirm only the matching gauge moves.

3. Enable Connector power saving: clients use their existing fallback values. Disable it: the cached state becomes available again.

4. Restart/reconfigure the hub: state stays unavailable until all five required values have arrived. Test unplugging the hub: client packet freshness expires after one second.

5. Optional: configure the three fake encoders above, then confirm remote control events in Connector and the resulting simulator feedback on the displays.

Connector sends changed values, so there is no timeout on unchanged simulator values. A Connector stop/USB disconnect is not inferred from output silence; explicit power-saving or hub detach marks the state unavailable. Hardware tests are needed to establish Connector stop behavior. Packet freshness detects loss of the hub radio, not a frozen PC bridge.